

COMP 8005
Assignment 2
Design

Manraj Bhullar

A01018456

February 15, 2026

Overview

The purpose of this document is to detail the design for the next iteration of the distributed UNIX password cracking system. The system has evolved from its baseline version to move away from a single-threaded model to a multi-threaded system. Additionally, a heartbeat protocol has also been added to provide liveness and visibility of progress during jobs.

While the previous version was strictly single-threaded, the worker now performs password cracking using multiple threads to speed up the process. The worker tests candidates across all active threads with no duplicates or omissions. The controller also periodically receives a heartbeat from the worker without interrupting the ongoing cracking process.

Controller Design

Goal

The goal of the controller is to coordinate the password cracking process by reading and parsing the UNIX shadow file, assigning the cracking job to a worker, and collecting the final results. In this iteration, the goal is expanded to include active monitoring of the worker through a heartbeat protocol.

Success

The controller is considered successful when it correctly parses the shadow entry, registers a worker, and dispatches the cracking job. It must then successfully manage the heartbeat cycle to receive updates from the worker and finally report results.

Invariant

The controller must not dispatch a cracking job unless a worker has successfully registered and the shadow file has been parsed correctly. Once the job is active, the controller must strictly maintain the heartbeat interval to ensure consistent tracking of progress without corrupting the search or missing updates.

Controller Responsibilities

1. Accepting a shadow file, username, port, and heartbeat interval from the command line.
2. Parsing the shadow file to extract the hashing algorithm identifier, salt, and password hash.
3. Accepting a single worker registration.
4. Sending the full cracking job to the worker.
5. Managing periodic heartbeats and reporting progress.
6. Reporting timing breakdowns and final results.

Testing Strategy

Test 1: Heartbeat Interval Validation

Steps

1. Run the controller with missing, empty, non-integer, negative, and valid interval values.

Expected Result

- For invalid inputs, the controller prints a usage or error message and exits cleanly.
- For a valid heartbeat interval, the controller starts normally and listens for worker connections.

What it Proves

- The controller correctly validates the heartbeat interval argument and prevents invalid configurations.

Test 2: Heartbeat Interval Adjust

Steps

1. Run the controller with different heartbeat intervals.
2. Observe heartbeat frequency during the same cracking job.

Expected Result

- Shorter intervals produce more frequent heartbeat logs.
- Longer intervals produce fewer responses.

What it Proves

- The heartbeat protocol follows the configured interval and functions correctly.

Test 3: Missed Heartbeat Response & Retries

Steps

1. Add a sleep command at the start of the worker's crack state.
2. Run the system with different heartbeat intervals to produce misses and observe.

Expected Result

- With a longer heartbeat interval, the controller temporarily misses heartbeats but continues execution once a response is received before exceeding the allowed miss threshold.
- With a shorter heartbeat interval, the miss threshold is exceeded before a response is received and the controller declares the worker unresponsive.

What it Proves

- The controller correctly detects missed heartbeats and enforces the maximum miss threshold for heartbeat responses from the worker.

Note: Actual testing breaks these tests down further. Please refer to the testing document.

Controller FSM

State Meanings

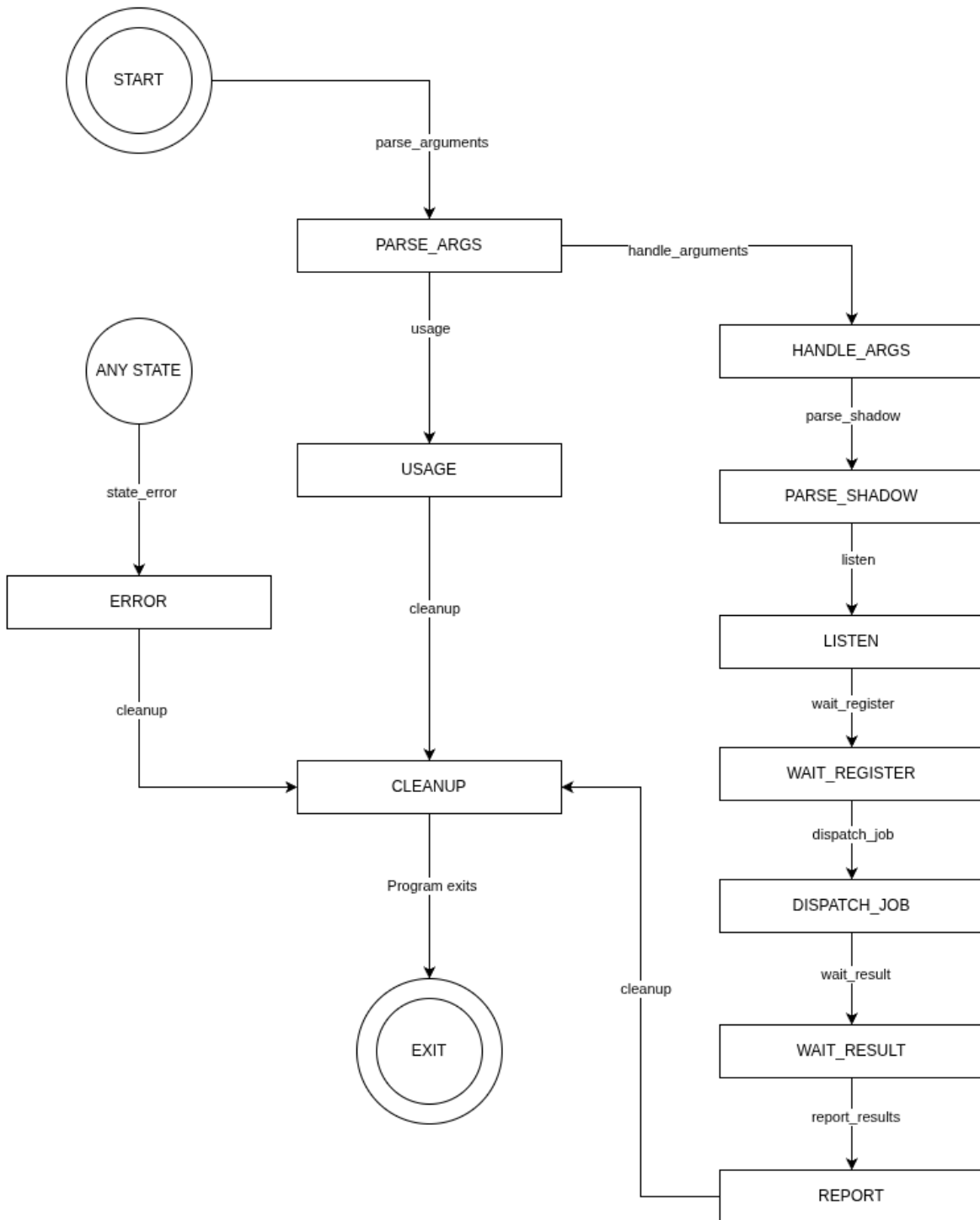
State	Meaning
START	Program is ran.
PARSE_ARGS	Command-line arguments get parsed.
HANDLE_ARGS	Parsed arguments are validated and prepared for use.
PARSE_SHADOW	Shadow file is read and the username's entry is parsed to extract algorithm, salt, and hash.
LISTEN	Controller waits for a worker connection.
WAIT_REGISTER	Controller waits and accepts worker registration.
DISPATCH_JOB	Controller sends cracking job to the worker.

WAIT_RESULT	Controller waits for the worker while managing the heartbeat cycle and monitoring progress.
REPORT	Controller prints the final result and timing breakdown.
USAGE	Command line error message displays if program is ran incorrectly.
ERROR	Reports when state error occurs and redirects to CLEANUP.
CLEANUP	Program cleans up resources.
EXIT	Program exits.

State Transition Table

Current State	Call (function)	Fail State	Success State
START	parse_arguments		PARSE_ARGS
PARSE_ARGS	handle_arguments	USAGE	HANDLE_ARGS
HANDLE_ARGS	parse_shadow	USAGE	PARSE_SHADOW
PARSE_SHADOW	listen	ERROR	LISTEN
LISTEN	accept_worker	ERROR	WAIT_REGISTER
WAIT_REGISTER	receive_registration	ERROR	DISPATCH_JOB
DISPATCH_JOB	send_job	ERROR	WAIT_RESULT
WAIT_RESULT	monitor_and_receive	ERROR	REPORT
REPORT	cleanup	ERROR	CLEANUP
USAGE	usage		CLEANUP
ERROR	state_error		CLEANUP
CLEANUP			EXIT

FSM Diagram



Functions

Name	Inputs	Outputs	Responsibility
parse_arguments	Command-line inputs	Parsed argument data	Parse command-line arguments.
handle_arguments	Parsed arguments	Validated arguments	Validate arguments and prepare for use.
parse_shadow	Shadow file path and username	Password data	Read shadow file and extract correct data.
listen	IP address and port number	Listening socket	Setup controller to listen for workers.
accept_worker	Listening socket	Connection info	Accept a worker connection.
receive_registration	Worker connection	Registration status	Registers worker.
send_job	Worker connection and job data	Job status	Send cracking job to worker.
wait_result	Worker connection	Heartbeat updates and results data	Manages heartbeats, receives cracking results, and reports.
usage	Error context	Error display message	Display usage message.
report_results	Results data	Final cracking results	Print final cracking results
state_error	Exit message	Exit message	Handles state error
cleanup	Program context	Exit status	Closes resources and terminate cleanly.

Worker Design

Goal

The goal of the worker is to receive a password cracking job from the controller and attempt to crack the encrypted password by computing and testing all possible candidates using multiple threads. In this iteration, the worker must also provide real-time progress updates by sending periodic heartbeats to the controller with progress updates since the last interval.

Success

The worker is considered successful when it completes the assigned cracking job. It must successfully spread the work across the configured number of threads, handle heartbeat responses simultaneously, and finally send the final results back to the controller when cracking completes.

Invariant

The worker must ensure that every candidate in the search space is tested exactly once across all active threads with no duplicates or omissions. It must also remain responsive to heartbeat requests without interrupting the cracking process.

Worker Responsibilities

- Starting a service and registering with the controller.
- Receiving a cracking job and the specified number of threads from the controller.
- Conducting password cracking using multiple threads.
- Responding to periodic heartbeat requests.
- Returning the cracked password or failure status along with final timing data to the controller.

Testing Strategy

Test 1: Thread Count Validation

Steps

1. Run the worker with missing, non-integer, negative, and valid -t values.

Expected Result

- Invalid inputs cause the worker to print an error and exit cleanly.
- A valid thread count allows successful registration with the controller.

What it Proves

- The worker properly validates the thread count argument.

Test 2: Multithreading Correctness

Steps

1. Run the system with a low amount of threads.
2. Run again with more threads using the same password.

Expected Result

- Both runs complete successfully.
- The total number of attempts remains identical.
- Hashes per second increase as threads increase and E2E runtime decreases.

What it Proves

- The worker effectively uses multithreading to improve performance.

Note: Actual testing breaks these tests down further. Please refer to the testing document.

Worker FSM

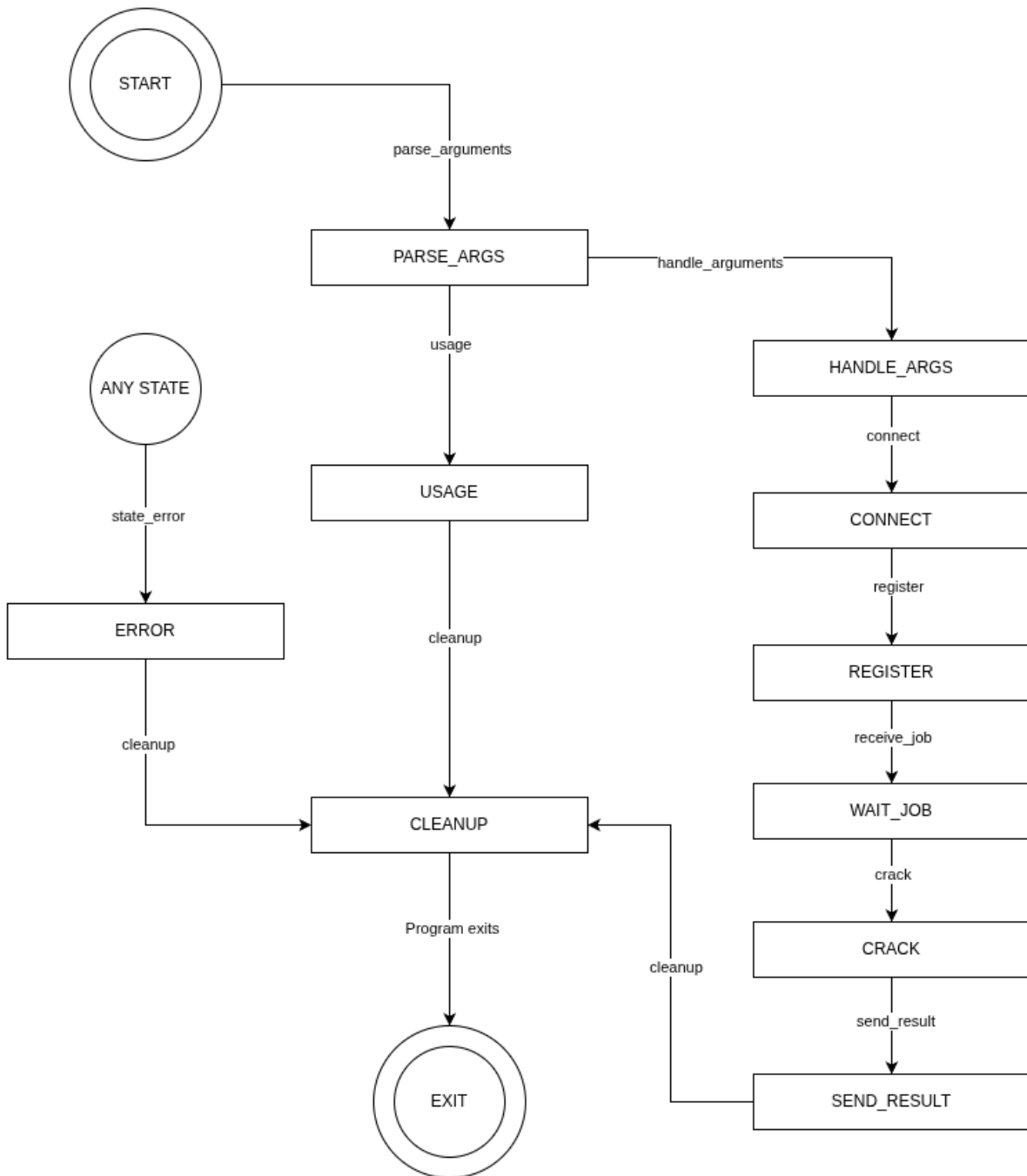
State Meanings

State	Meaning
START	Program is ran.
PARSE_ARGS	Command-line arguments get parsed.
HANDLE_ARGS	Parsed arguments are validated and prepared for use.
CONNECT	Worker connects to the controller.
REGISTER	Worker registers with controller.
WAIT_JOB	Worker waits for job from controller.
CRACK	Worker performs cracking job while responding to heartbeat requests.
SEND_RESULT	Sends cracking result to controller.
USAGE	Error message displays if program ran incorrectly.
ERROR	Reports when state error occurs and redirects to CLEANUP.
CLEANUP	Program cleans up resources.
EXIT	Program exits.

State Transition Table

Current State	Call (function)	Fail State	Success State
START	parse_arguments		PARSE_ARGS
PARSE_ARGS	handle_arguments	USAGE	HANDLE_ARGS
HANDLE_ARGS	connect	USAGE	CONNECT
CONNECT	register	ERROR	REGISTER
REGISTER	receive_job	ERROR	WAIT_JOB
WAIT_JOB	crack_and_respond	ERROR	CRACKING
CRACK	send_result	ERROR	SEND_RESULT
SEND_RESULT	cleanup	ERROR	CLEANUP
USAGE	usage		CLEANUP
ERROR	state_error		CLEANUP
CLEANUP			EXIT

FSM Diagram



Functions

Name	Inputs	Outputs	Responsibility
parse_arguments	Command-line inputs	Parsed argument data	Parse command-line arguments.
handle_arguments	Parsed arguments	Validated arguments	Validate arguments and prepare for use.
connect	Controller IP address and port	Connection status	Establish connection to controller.
register	Controller socket	Registration status	Register as worker.
receive_job	Controller socket and job info	Job status	Accept a job from controller.
crack_and_respond	Job info and thread count	Cracked password result and heartbeat responses	Process cracking job while responding to heartbeat requests from controller.
send_result	Controller socket and results data	Result and timing info	Send results to controller.
usage	Error context	Display message	Display usage message.
state_error	Exit message	Prints exit message	Notify what state error occurred.
cleanup	Program context	Exit status	Clean up resources and terminate cleanly.